МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение высшего образования

«ВЛАДИВОСТОКСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ»
(ФГБОУ ВО «ВВГУ»)

Институт информационных технологий и анализа данных
Кафедра информационных технологий и систем

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ

по дисциплине «Информатика и основы программирования»

Обработка последовательностей

Студент
гр. БИН-26-3 _____ В. В. Шокина

Преподаватель _____ М. В. Водяницкий

Владивосток 2026

1 Выполнение работы

1.1 Задание 1. Обратный отсчет

Пользователь вводит неотрицательное целое число n. С помощью цикла while необходимо вывести все целые числа от n до 0 включительно в порядке убывания, каждое на отдельной строке.

Блок-схема программы для задания 1 представлена на рисунке 1.

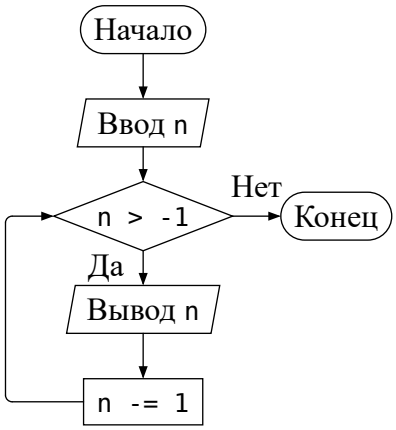


Рисунок 1 – Блок-схема программы для задания 1

Алгоритм начинается с ввода числа n. Затем проверяется условие n > −1. Пока оно истинно, число выводится на экран и уменьшается на единицу, после чего проверка повторяется. Когда условие становится ложным, алгоритм завершается.

Текст программы для задания 1 представлен на рисунке 2.

```
1  n = int(input())
2  while n > -1:
3      print(n)
4      n -= 1
```

Рисунок 2 – Текст программы для задания 1

В первой строке программа считывает число n и преобразует его в целое. Цикл while выполняется, пока n больше −1, то есть включает значение 0. В теле цикла функция print() выводит текущее значение n, а оператор n -= 1 уменьшает его на единицу. Когда n становится равным −1, цикл завершается.

Результат работы программы при вводе 3 представлен на рисунке 3.

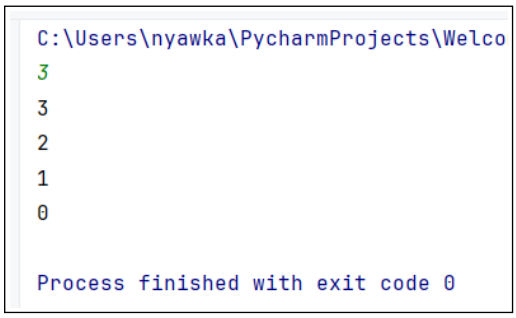


Рисунок 3 – Результат работы программы для задания 1

При вводе числа 3 программа вывела числа 3, 2, 1 и 0, каждое на отдельной строке, в порядке убывания. Задание выполнено.

1.2 Задание 2. Сумма и произведение

Пользователь вводит целые числа a и b на отдельных строках, причем a <= b. Необходимо вычислить сумму и произведение всех целых чисел от a до b включительно с помощью цикла for. Сумма и произведение вычисляются накоплением в цикле, без готовых функций и формул.

Блок-схема программы для задания 2 представлена на рисунке 4.

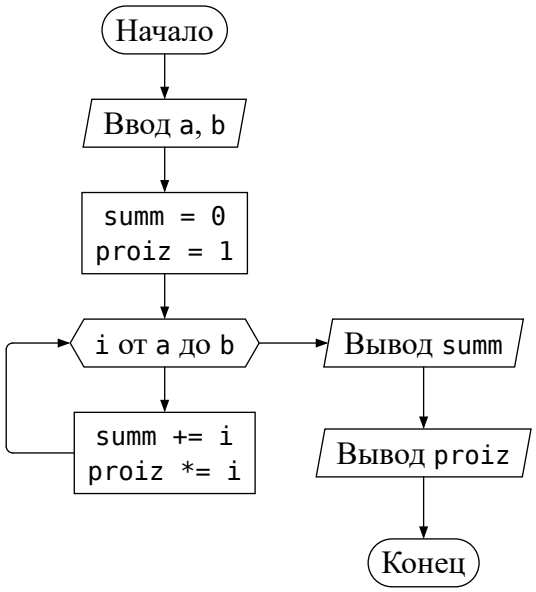


Рисунок 4 – Блок-схема программы для задания 2

Алгоритм начинается с ввода чисел a и b. Затем сумма обнуляется, а произведение принимает значение 1. В цикле для каждого i от a до b к сумме прибавляется i, а произведение умножается на i. После завершения цикла выводятся сумма и произведение, и алгоритм заканчивается.

Текст программы для задания 2 представлен на рисунке 5.

```python
a = int(input())
b = int(input())
summ = 0
proiz = 1
for i in range(a, b + 1):
    summ += i
    proiz *= i
print("Сумма:", summ)
print("Произведение:", proiz)
```

Рисунок 5 – Текст программы для задания 2

В первых двух строках программа считывает числа a и b. Переменная summ для суммы получает начальное значение 0, а переменная proiz для произведения получает значение 1. Цикл for перебирает все целые числа от a до b включительно, для этого в range() указано b + 1. На каждом шаге значение i добавляется к summ и умножается на proiz.

Результат работы программы при вводе 2 и 5 представлен на рисунке 6.

```
C:\Users\nyawka\PycharmProjects\Wel
2
5
Сумма: 14
Произведение: 120

Process finished with exit code 0
```

Рисунок 6 – Результат работы программы для задания 2

При вводе чисел 2 и 5 программа вычислила сумму 14 и произведение 120. Задание выполнено.

1.3 Задание 3. Подсчет измерений

Пользователь вводит неотрицательное целое число n, затем n значений температуры как вещественные числа, каждое на отдельной строке. Необходимо подсчитать количество измерений ниже нуля, равных нулю и выше нуля. Значения обрабатываются по одному, количества накапливаются в цикле. Если n равно 0, все три количества равны нулю.

Блок-схема программы для задания 3 представлена на рисунке 7.

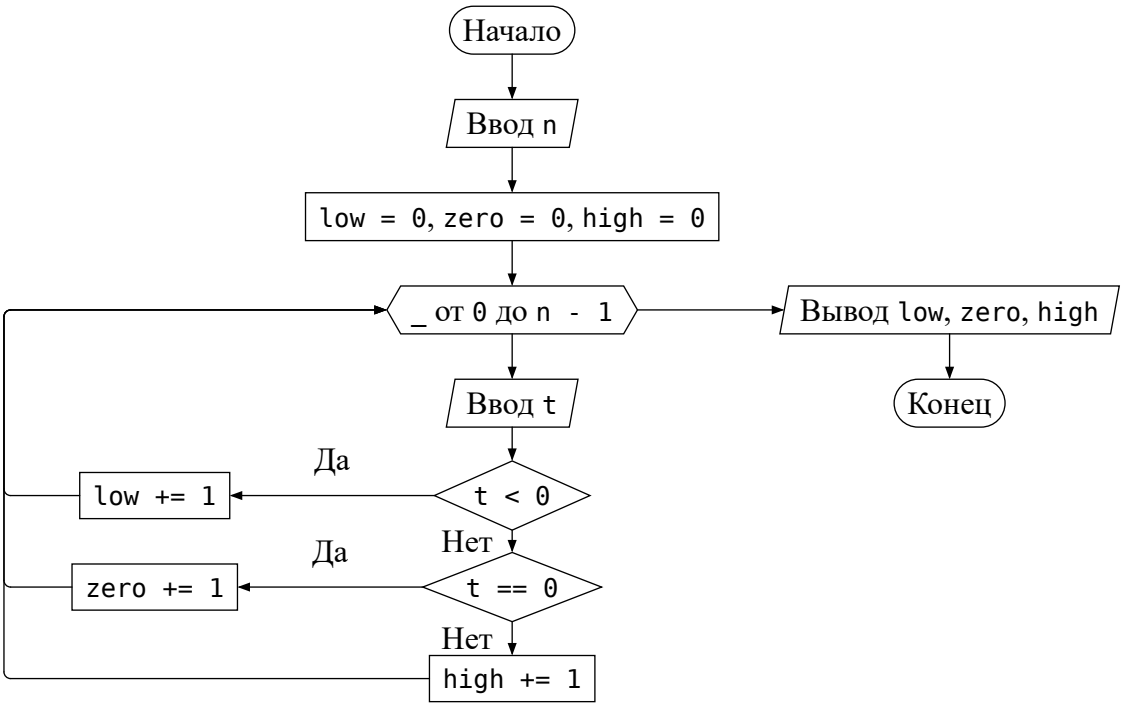


Рисунок 7 – Блок-схема программы для задания 3

Алгоритм начинается с ввода количества измерений n и обнуления трех счетчиков. Затем в цикле, который выполняется n раз, вводится очередное значение температуры t. С помощью двух условий оно сравнивается с нулем, и увеличивается один из счетчиков: отрицательных, нулевых или положительных значений. После завершения цикла значения счетчиков выводятся на экран.

Текст программы для задания 3 представлен на рисунке 8.

```
1   n = int(input())
2   low = 0
3   zero = 0
4   high = 0
5   for _ in range(n):
6       t = float(input())
7       if t < 0:
8           low += 1
9       elif t == 0:
10          zero += 1
11      else:
12          high += 1
13  print("Ниже нуля:", low)
14  print("Равны нулю:", zero)
15  print("Выше нуля:", high)
```

Рисунок 8 – Текст программы для задания 3

Программа считывает n и создает три счетчика: low, zero и high с начальным значением 0. На каждом шаге вводится вещественное число. Программа определяет, что значение меньше

нуля, равно нулю или больше нуля, и увеличивает соответствующий счетчик на единицу. Если n равно 0, цикл не выполняется, и все счетчики остаются нулевыми.

Результат работы программы при вводе 5 измерений (-2.5, 0, 3.1, −1, 0) представлен на рисунке 9.

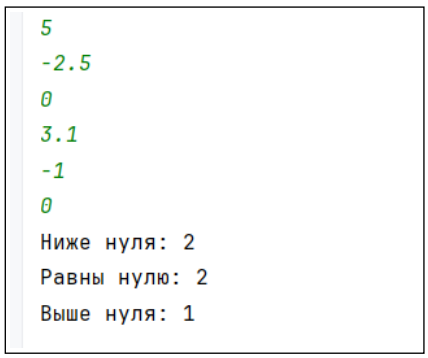


Рисунок 9 – Результат работы программы для задания 3

Среди введенных значений два отрицательных (-2.5 и −1), два нулевых и одно положительное (3.1). Задание выполнено.

1.4 Задание 4. Таблица умножения

Пользователь вводит целое число n от 1 до 9 включительно. Необходимо вывести таблицу умножения этого числа на целые числа от 1 до 10 включительно в порядке возрастания второго множителя.

Блок-схема программы для задания 4 представлена на рисунке 10.

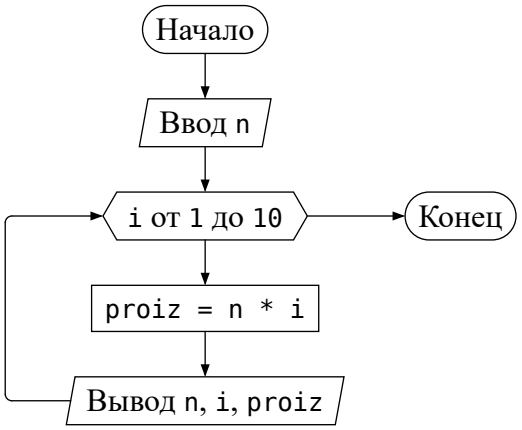


Рисунок 10 – Блок-схема программы для задания 4

Алгоритм начинается с ввода числа n. Затем в цикле для каждого множителя i от 1 до 10 вычисляется произведение n на i и выводится строка таблицы умножения. После выполнения всех повторений цикла алгоритм завершается.

Текст программы для задания 4 представлен на рисунке 11.

```
n = int(input())

for i in range(1,11):
    proiz = n * i
    print(f'{n} * {i} = {proiz}')
```

Рисунок 11 – Текст программы для задания 4

В первой строке программа считывает число n. Цикл for перебирает значения i от 1 до 10, для этого в range() указано 11. В теле цикла переменной proiz присваивается произведение n **i, а функция print() с f-строкой выводит строку в формате «3** 1 = 3».

Результат работы программы при вводе 3 представлен на рисунке 12.

```
3
3 * 1 = 3
3 * 2 = 6
3 * 3 = 9
3 * 4 = 12
3 * 5 = 15
3 * 6 = 18
3 * 7 = 21
3 * 8 = 24
3 * 9 = 27
3 * 10 = 30
```

Рисунок 12 – Результат работы программы для задания 4

При вводе числа 3 программа вывела десять строк таблицы умножения от «3 **1 = 3» до «3** 10 = 30» в порядке возрастания второго множителя. Задание выполнено.

1.5 Задание 5. Учет покупок

Пользователь последовательно вводит стоимость каждой покупки в целых рублях на отдельной строке. Количество покупок заранее неизвестно, ввод завершается строкой stop, которая не является покупкой. Необходимо вывести количество покупок, общую стоимость и среднюю стоимость покупки с двумя цифрами после десятичной точки. Если команда stop введена до первой покупки, выводится только сообщение «Нет покупок».

Блок-схема программы для задания 5 представлена на рисунке 13.

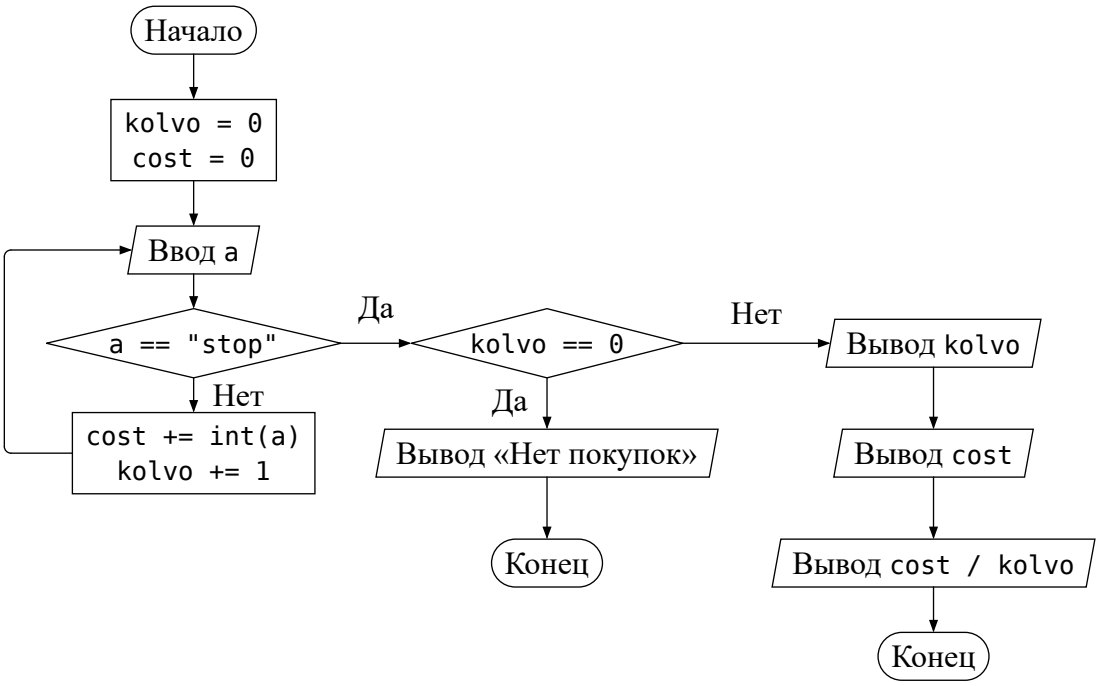


Рисунок 13 – Блок-схема программы для задания 5

Алгоритм начинается с обнуления счетчика покупок и общей стоимости. Затем в бесконечном цикле вводится строка a. Если это строка stop, цикл прерывается. В противном случае стоимость прибавляется к общей сумме, а счетчик покупок увеличивается на единицу. После выхода из цикла проверяется, были ли покупки. Если их не было, выводится сообщение «Нет покупок», иначе выводятся количество, общая и средняя стоимость.

Текст программы для задания 5 представлен на рисунке 14.

```
kolvo = 0
cost = 0
while True:
    a = input()
    if a == "stop":
        break
    cost += int(a)
    kolvo += 1
if kolvo == 0:
    print("Нет покупок")
else:
    print(f"Покупок: {kolvo}")
    print(f"Общая стоимость: {cost} руб")
    print(f"Средняя стоимость: {cost / kolvo:.2f} руб")

```

Рисунок 14 – Текст программы для задания 5

Переменные kolvo и cost хранят количество покупок и общую стоимость, их начальные значения равны 0. Цикл while True повторяется до команды stop. Внутри цикла строка читается функцией input(), и при значении «stop» оператор break прерывает цикл. Иначе строка преобразуется в целое число и прибавляется к cost, а kolvo увеличивается на единицу. После

цикла условие kolvo == 0 защищает от деления на ноль и выводит сообщение «Нет покупок». В остальных случаях программа выводит количество покупок, общую стоимость и среднюю стоимость cost / kolvo с форматом :.2f, то есть с двумя знаками после точки.

Результат работы программы при вводе 120, 85, 200 и stop представлен на рисунке 15.

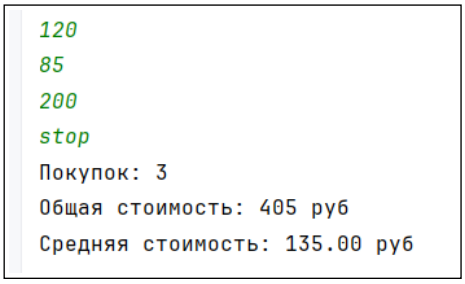


Рисунок 15 – Результат работы программы для задания 5

Для трех покупок общая стоимость составила 120 + 85 + 200 = 405 руб, а средняя стоимость 405 / 3 = 135.00 руб. Задание выполнено.

1.6 Задание 6. Треугольник Паскаля

Пользователь вводит количество строк n как целое число от 1 до 15 включительно. Необходимо вывести первые n строк треугольника Паскаля, начиная с вершины, после чего вывести количество нечетных элементов во всех выведенных строках. Элементы строки вычисляются во вложенных циклах по соотношению `C(r, k + 1) = C(r, k) * (r - k) // (k + 1)`.

Блок-схема программы для задания 6 представлена на рисунке 16.

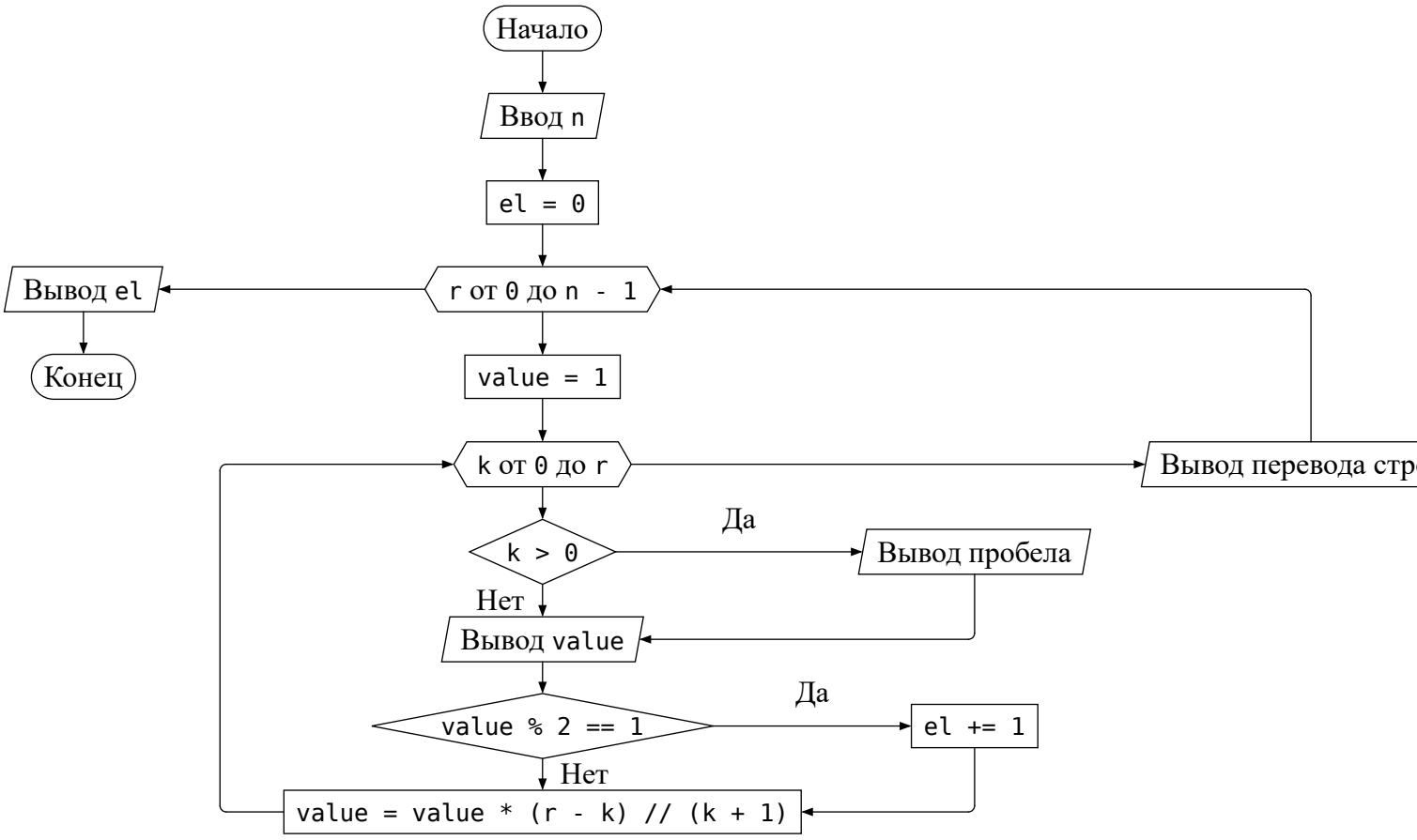


Рисунок 16 – Блок-схема программы для задания 6

Алгоритм начинается с ввода числа строк n и обнуления счетчика нечетных элементов. Внешний цикл перебирает номера строк r. Для каждой строки значение первого элемента равно 1, а внутренний цикл перебирает позиции k в строке. Внутри него выводится пробел перед всеми элементами, кроме первого, затем выводится сам элемент. Если элемент нечетный, счетчик увеличивается. После этого вычисляется следующий элемент строки. После внутреннего цикла происходит перевод строки. По окончании внешнего цикла выводится количество нечетных элементов.

Текст программы для задания 6 представлен на рисунке 17.

```python
n = int(input())
el = 0
for r in range(n):
    value = 1
    for k in range(r + 1):
        if k > 0:
            print(' ', end='')
        print(value, end='')
        if value % 2 == 1:
            el += 1
        value = value * (r - k) // (k + 1)
    print()
print(f'Нечетных элементов: {el}')
```

Рисунок 17 – Текст программы для задания 6

Программа считывает n и создает счетчик el для нечетных элементов. Внешний цикл по r формирует строки треугольника, в каждой из которых r + 1 элемент. В начале каждой строки переменная value равна 1. Внутренний цикл по k выводит элементы строки через функцию print() с параметром end=„“, чтобы они оставались на одной строке. Перед каждым элементом, кроме первого, выводится один пробел. Следующий элемент вычисляется по формуле `value * (r - k) // (k + 1)` без списков и готовых функций. После внутреннего цикла print() переводит строку, а в конце программы выводится значение el.

Результат работы программы при вводе 5 представлен на рисунке 18.

```
5
1
1 1
1 2 1
1 3 3 1
1 4 6 4 1
Нечетных элементов: 11
```

Рисунок 18 – Результат работы программы для задания 6

При вводе числа 5 программа вывела пять строк треугольника Паскаля от «1» до «1 4 6 4 1». Количество нечетных элементов во всех строках равно 11. Задание выполнено.